



RÉGION ACADÉMIQUE
PAYS DE LA LOIRE

Liberté
Égalité
Fraternité



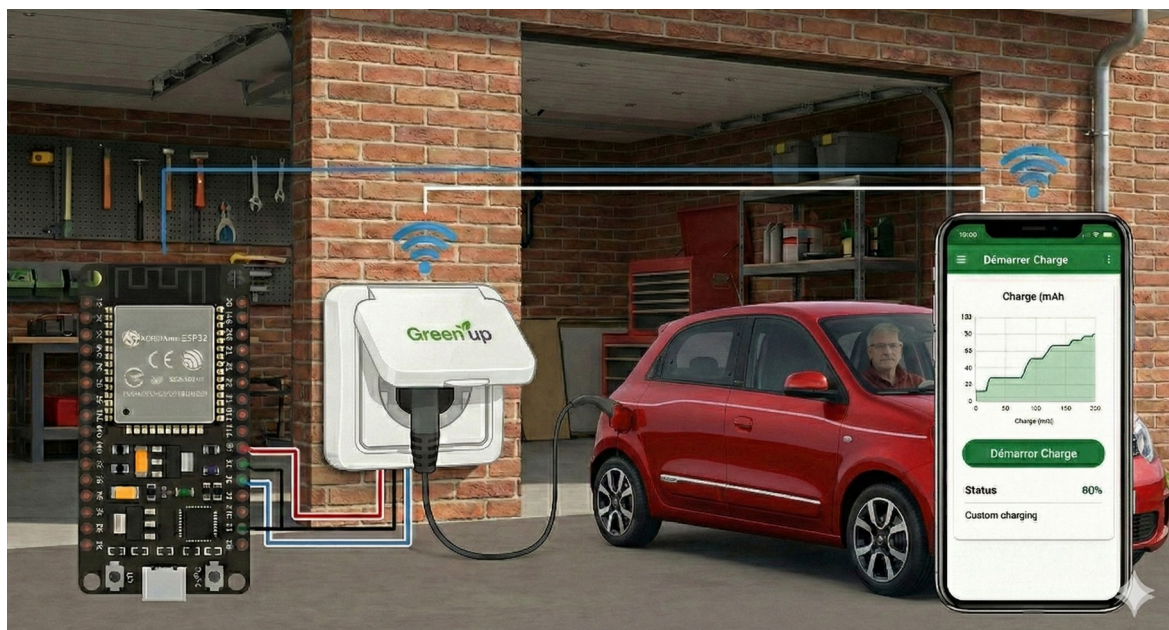
Lycée
**TOUCHARD
WASHINGTON**

Dossier Personnel

BTS CIEL SESSION 2025 - IR

E6 – PROJET TECHNIQUE

Gestionnaire de charge de Véhicules électriques



Lycée : Touchard-Washington		Ville : LE MANS	
Nom du projet :	Gestionnaire de charge de Véhicules électriques		
N° du projet :	TW2		
Projet nouveau	Oui ☒ Non ☐	Projet interne	Oui ☒ Non ☐
Délai de réalisation	150 heures	Statut des étudiants	Formation initiale ☒ Apprentissage ☐
Spécialité des étudiants	ER ☐ IR ☒ Mixte ☐	Nombre d'étudiants :	4 étudiants
Professeurs responsables	Philippe CRUCHET , Didier Bernard, François Martin, Anthony LE CREN, Jilali KHAMACH, François QUEREC		

Table des matières

1 Sélectionner et configurer la borne.....	3
1.1 Description du cas d'utilisation.....	3
1.2 IHM.....	4
1.3 Diagramme de séquences.....	7
2 Saisir le kilométrage du véhicule.....	8
2.1 Description du cas d'utilisation.....	8
2.2 IHM	9
2.3 Diagramme de séquence.....	10
3 Visualiser l'état du système.....	11
3.1 Description du cas d'utilisation.....	11
3.2 IHM.....	12
3.3 Diagramme de séquence.....	13
4. Recevoir des notifications.....	14
4.1 Description du cas d'utilisation.....	14
4.2 IHM.....	15
4.3 Diagramme de séquence.....	16
5. Protocoles.....	17
5.1 JSON.....	17
Trames échangées avec l'ESP32 :.....	17
Trames échangées avec le Raspberry Pi :.....	18
5.2 WebSocket.....	18
6. Outils.....	19
6.1 Qt Creator.....	19
3.2 QML.....	19
6.3 GitHub.....	20
7. Architecture de l'application.....	20
8. Diagramme de classe.....	21
9. Fiche de test.....	23
10. Diagramme de Gant.....	26
11. Compétences acquises.....	27
12. Perspectives d'évolution.....	28

1 Sélectionner et configurer la borne

1.1 Description du cas d'utilisation

Description du cas d'utilisation	Ce cas d'utilisation permet au conducteur de se connecter au système de gestion de charge. L'application affiche la liste des bornes de recharge disponibles. Le conducteur sélectionne la borne souhaitée pour commencer une session de charge, la modifier ou la supprimer. Il peut également démarrer ou arrêter une charge manuellement.
Pré-conditions	Le conducteur dispose d'un smartphone avec l'application installée. Au moins une borne est disponible dans le système.
Post-conditions	La borne est sélectionnée et active. Le conducteur peut accéder aux fonctionnalités de gestion de charge.
Scénario principal	L'application affiche la liste des bornes disponibles avec leur statut. Le conducteur sélectionne la borne souhaitée. L'application confirme la connexion à la borne et donne accès aux fonctionnalités.
Scénario alternatif	Si aucune borne n'est disponible, la liste reste vide et le conducteur ne peut pas démarrer de session.

1.2 IHM

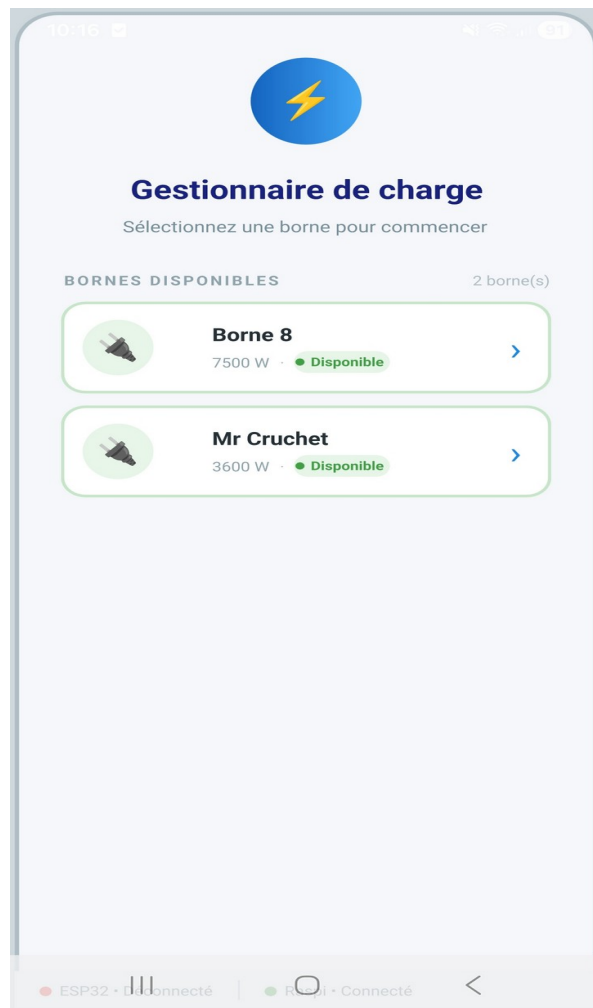


Figure 1: Page d'accueil de l'application

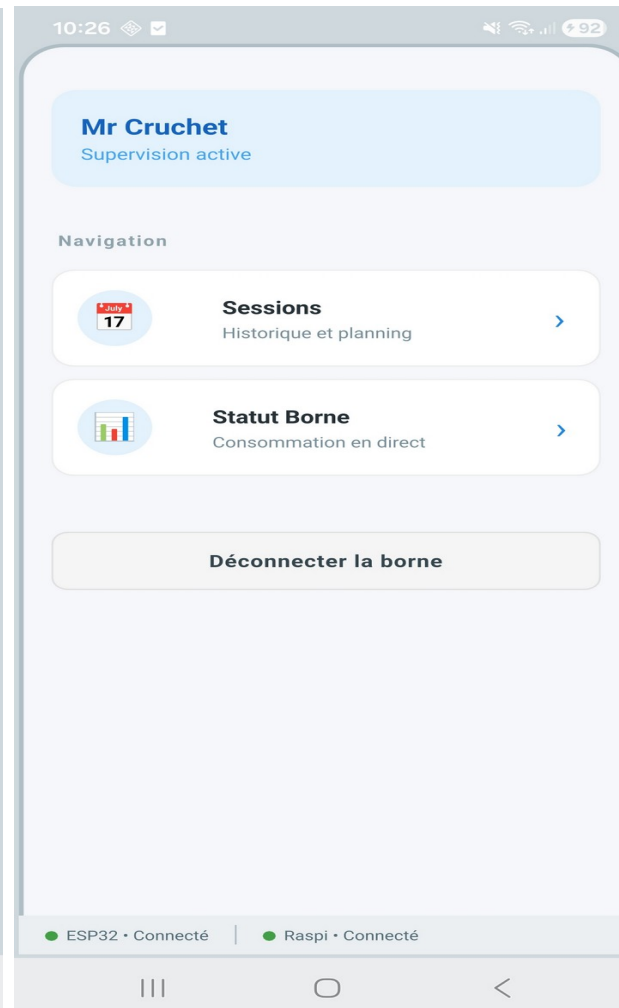


Figure 2: Tableau de bord de l'application

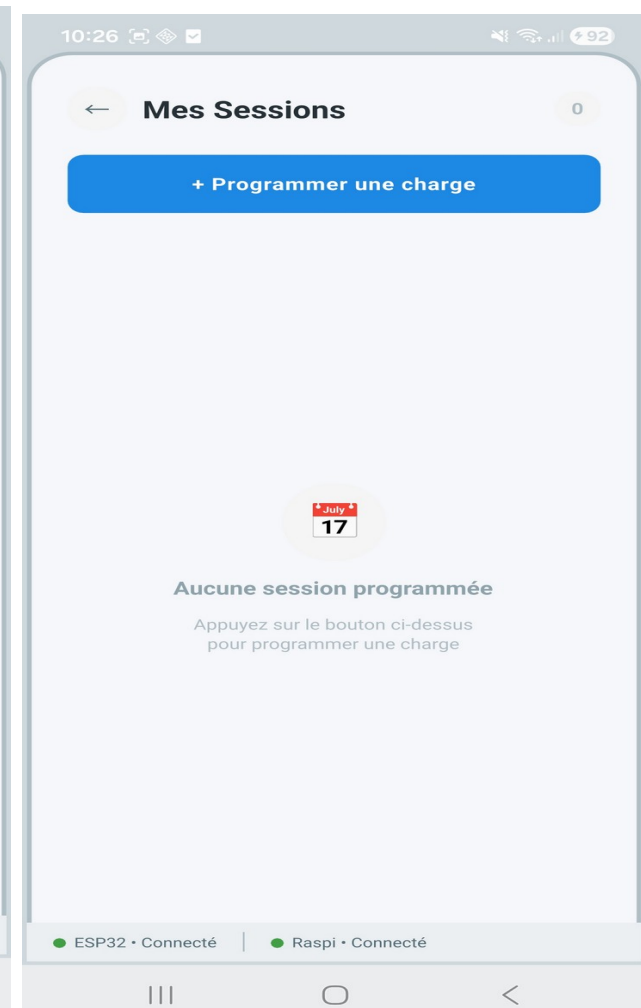


Figure 3: Page des sessions

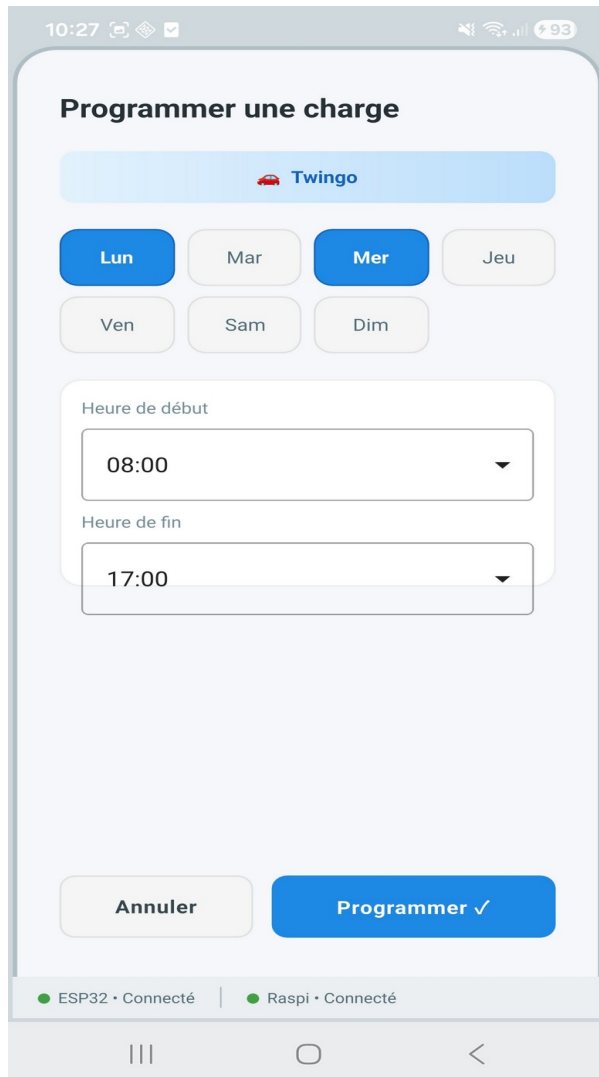


Figure 4: Programmation d'une session

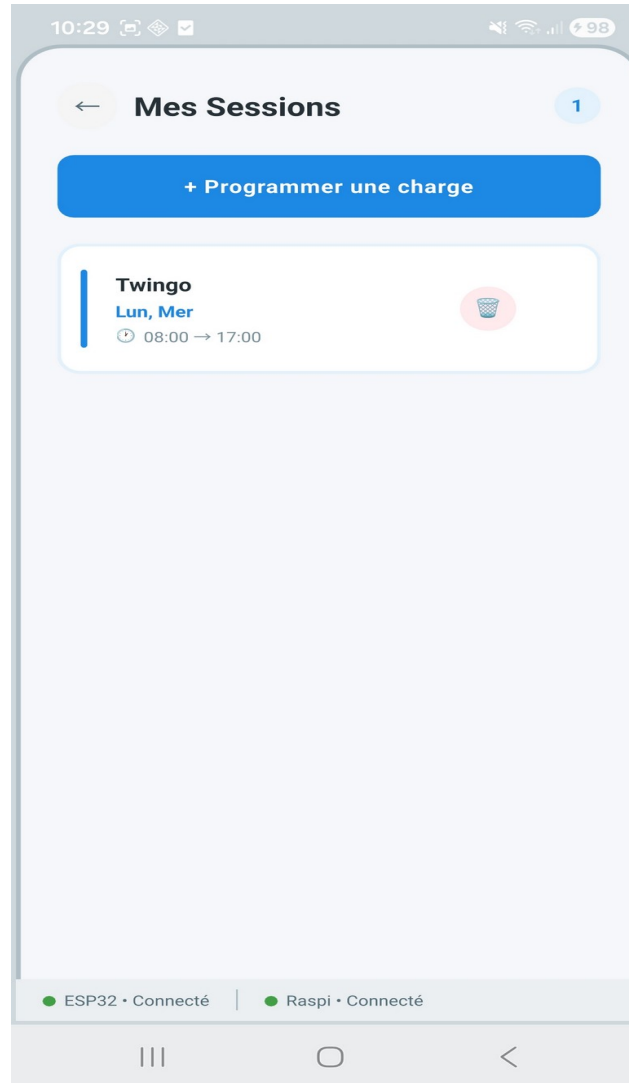


Figure 5: Affichage des sessions programmées

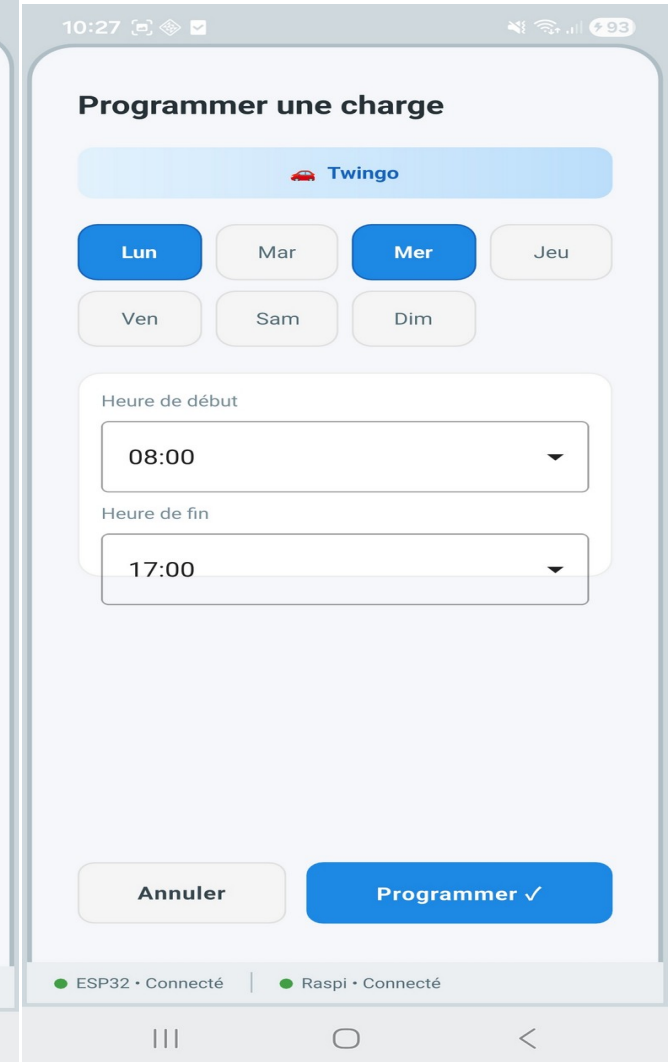


Figure 6: Modification d'une session de charge

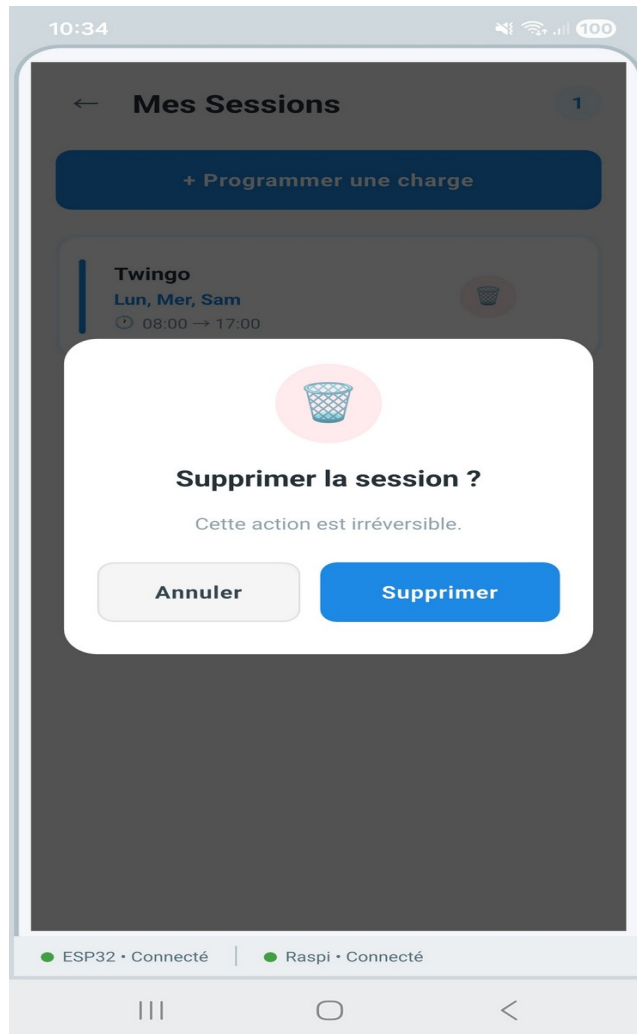


Figure 7: Suppression d'une session

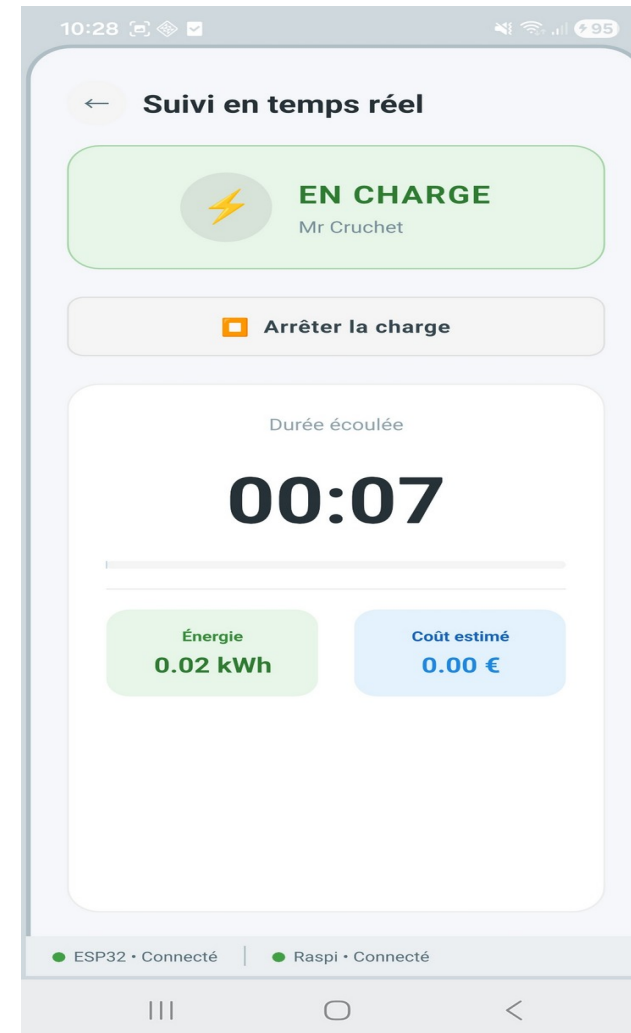


Figure 8: Mise en marche forcée

1.3 Diagramme de séquences

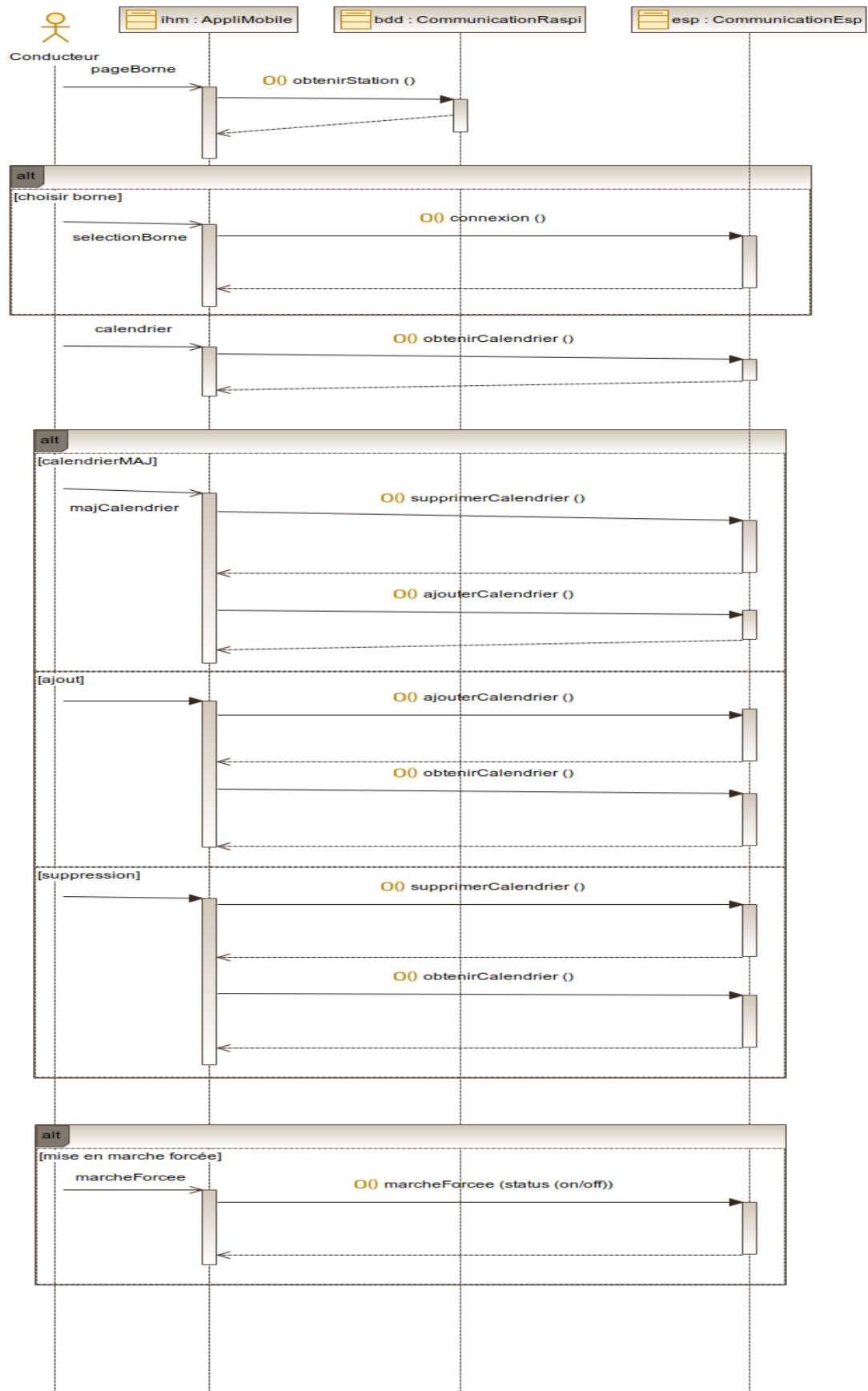


Figure 9: Diagramme de séquence : sélectionner la borne de recharge et configurer la borne

2 Saisir le kilométrage du véhicule

2.1 Description du cas d'utilisation

Description du cas d'utilisation	Ce cas d'utilisation permet au conducteur de gérer ses véhicules. Il peut consulter la liste des véhicules enregistrés, en ajouter un nouveau, sélectionner un véhicule existant et mettre à jour son kilométrage.
Pré-conditions	Une borne a été sélectionnée au préalable.
Post-conditions	Un véhicule est sélectionné et son kilométrage est mis à jour dans le système.
Scénario principal	L'application affiche la liste des véhicules enregistrés. Le conducteur sélectionne son véhicule ou en ajoute un nouveau. Il saisit le kilométrage actuel et valide. Le système enregistre les informations.
Scénario alternatif	Si le kilométrage saisi est inférieur à l'ancien relevé, le système affiche un message d'erreur et empêche la validation. Si aucun véhicule n'est enregistré, le conducteur est invité à en créer un.

2.2 IHM

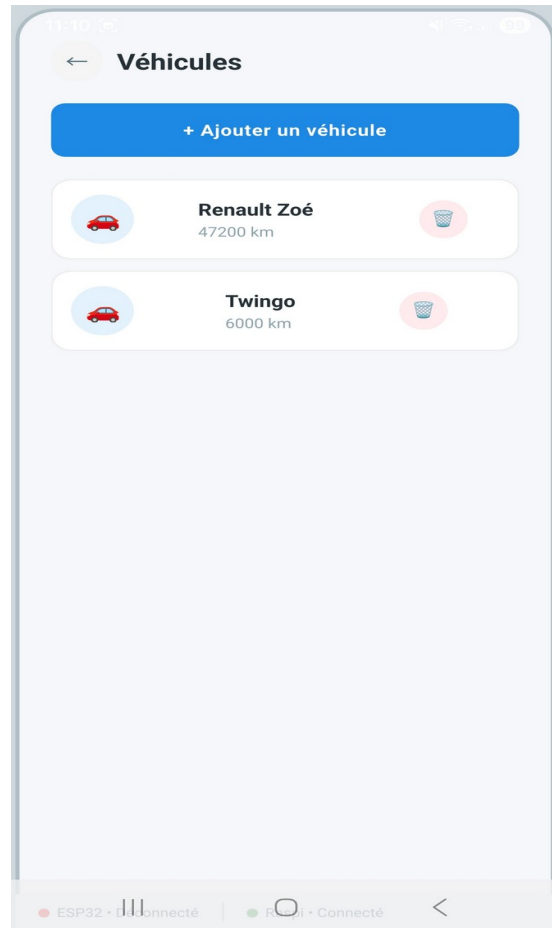


Figure 10: Affichages des véhicules

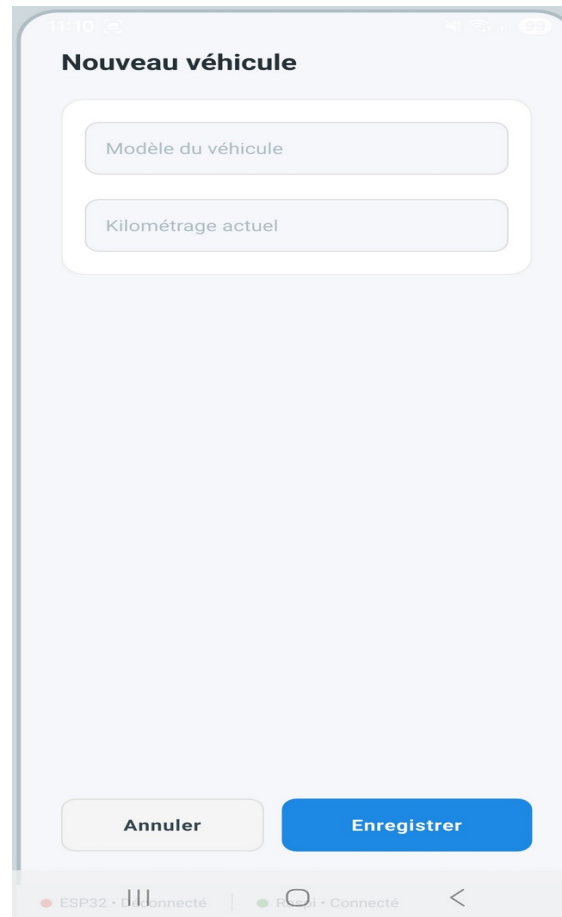


Figure 11: Ajout d'un véhicule

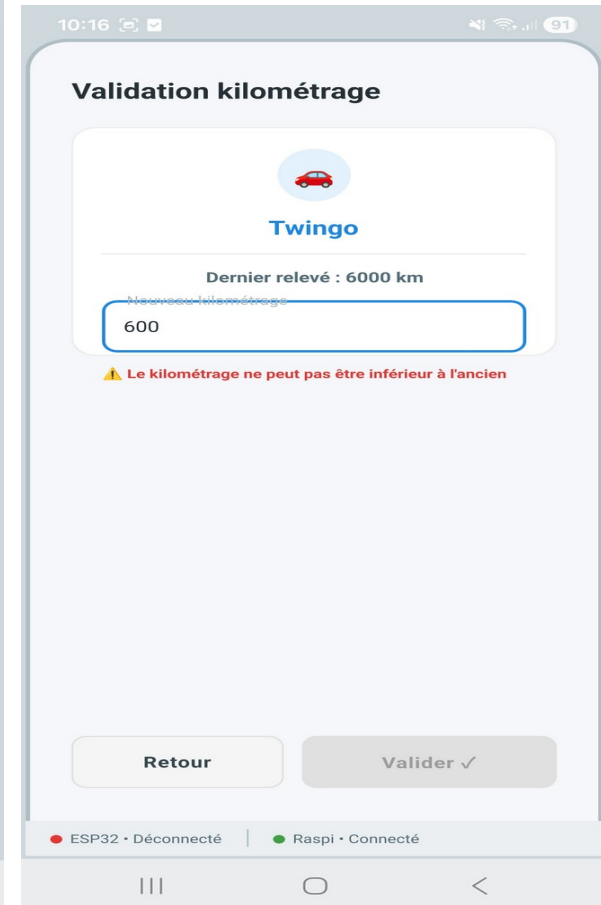


Figure 12: Mise à jour du kilométrage

2.3 Diagramme de séquence



Figure 13: Diagramme de séquence : Ajouter un véhicule / Saisir le kilométrage du véhicule

3 Visualiser l'état du système

3.1 Description du cas d'utilisation

Description du cas d'utilisation	Ce cas d'utilisation permet au conducteur de consulter en temps réel l'état de la borne de charge sélectionnée.
Pré-conditions	Une borne et un véhicule ont été sélectionnés au préalable.
Post-conditions	Le conducteur visualise l'état en temps réel de la borne et peut contrôler la charge.
Scénario principal	L'application affiche le statut de la borne (programmé, en charge), la durée de session écoulée, l'énergie consommée et le coût estimé.
Scénario alternatif	Si la connexion à la borne est perdue, les données ne se mettent plus à jour et un indicateur signale la déconnexion.

3.2 IHM

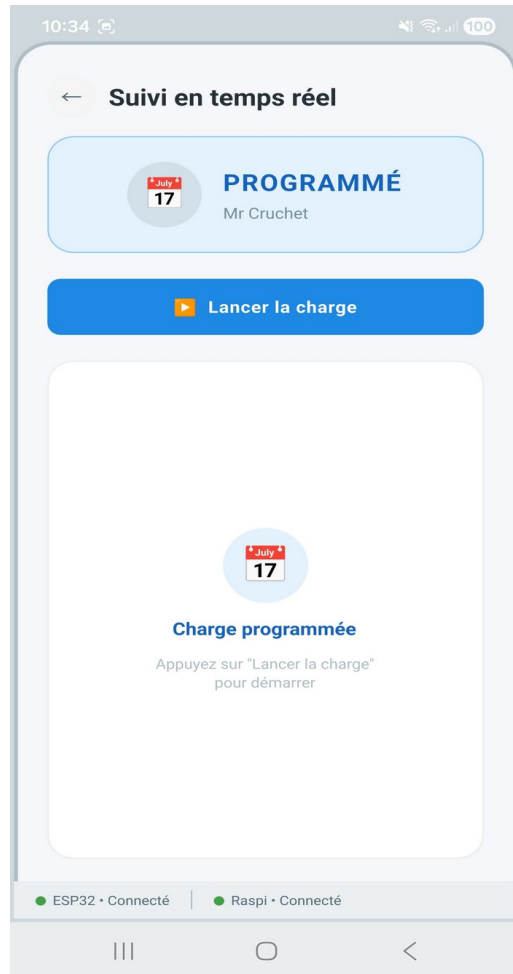


Figure 14: Status de la borne : programmé

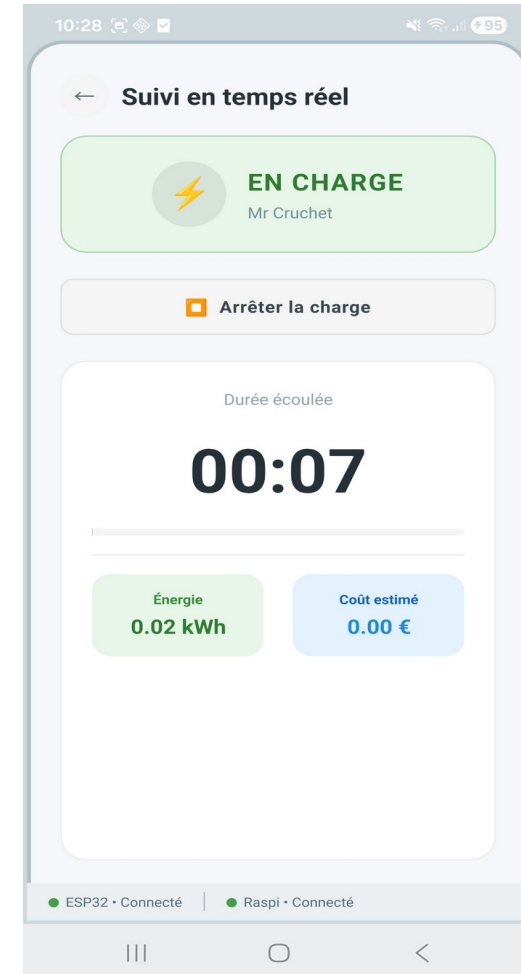


Figure 15: Status de la borne : en charge

3.3 Diagramme de séquence

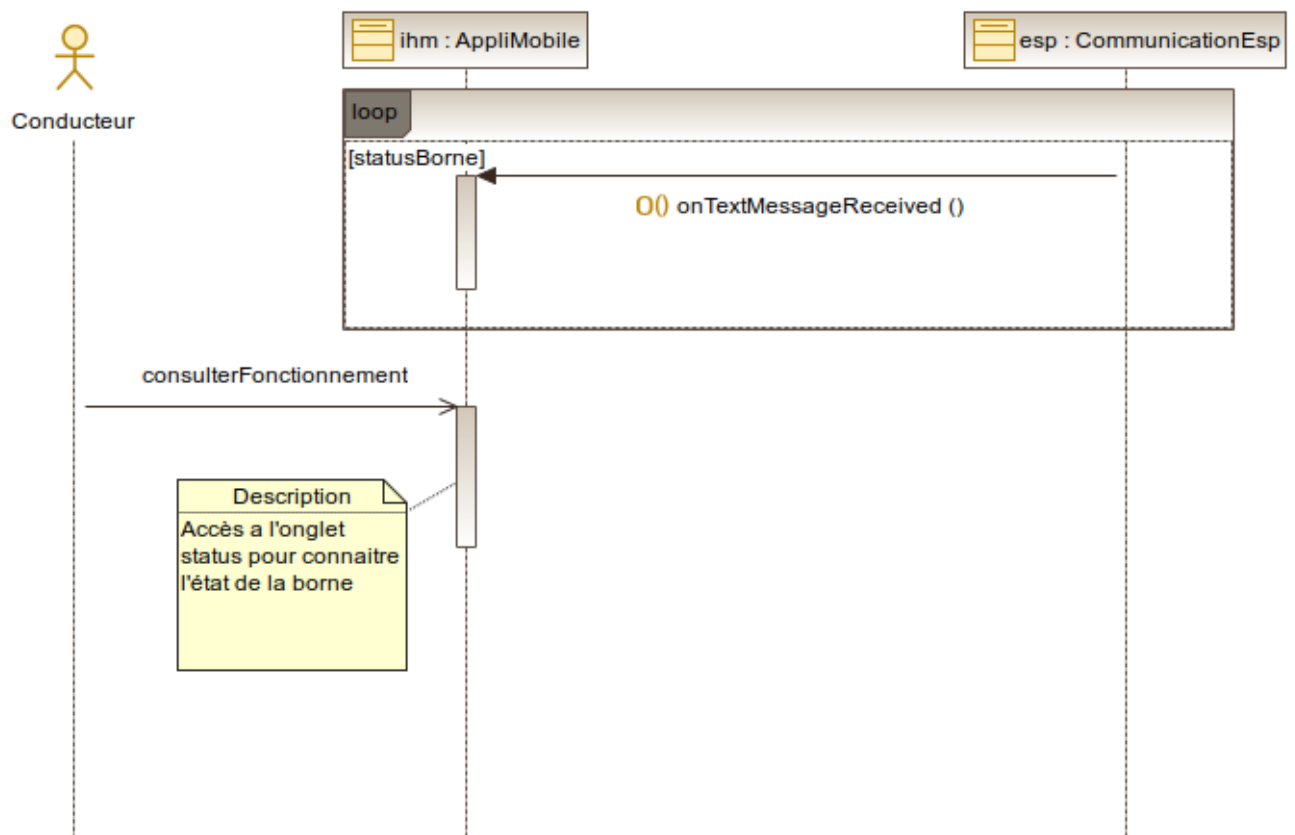


Figure 16: Diagramme de séquence : Recevoir des notifications

4. Recevoir des notifications

4.1 Description du cas d'utilisation

Description du cas d'utilisation	Ce cas d'utilisation permet à l'application de recevoir et d'afficher les alertes émises par la borne en cas d'anomalie détectée, comme une surchauffe. La notification s'affiche automatiquement quelle que soit la page active.
Pré-conditions	Une borne est active et en cours de supervision.
Post-conditions	L'alerte est affichée et acquittée par le conducteur.
Scénario principal	La borne détecte une anomalie et envoie une alerte à l'application. Une fenêtre s'affiche en premier plan indiquant la nature du problème. Le conducteur appuie sur "J'ai compris" pour acquitter l'alerte. La fenêtre se ferme.
Scénario alternatif	Si la connexion est perdue avant l'acquiescement, l'alerte reste affichée jusqu'à ce que le conducteur l'acquiesce manuellement.

4.2 IHM

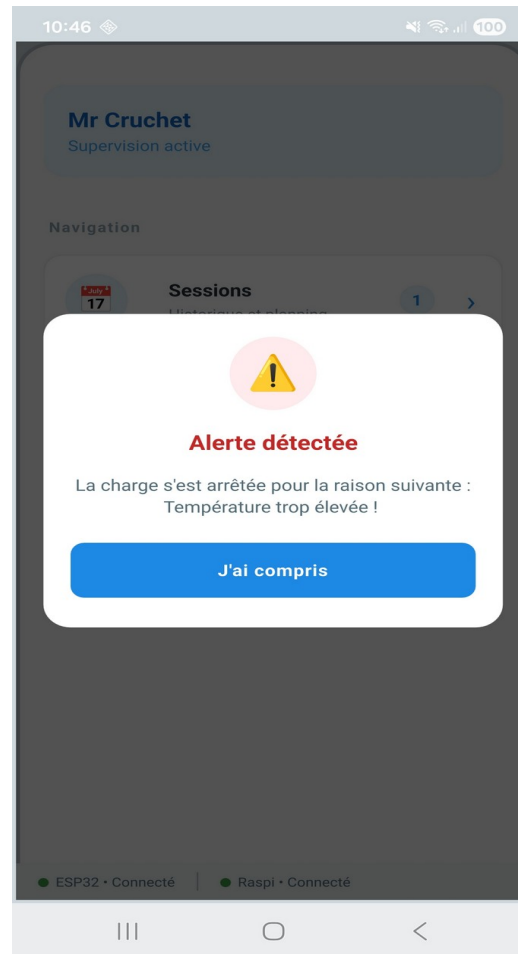


Figure 17: Alerte de surchauffe

4.3 Diagramme de séquence

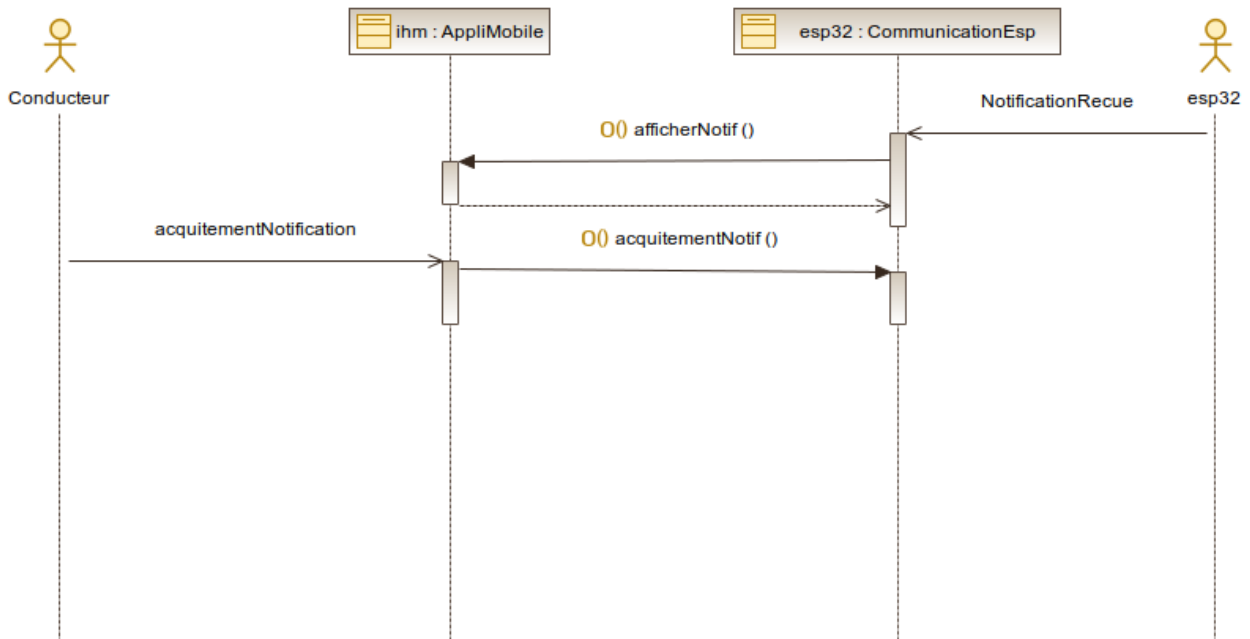


Figure 18: Diagramme de séquence : Visualiser l'état du système

5. Protocoles

5.1 JSON

Le format JSON (JavaScript Object Notation) a été retenu pour l'ensemble des échanges entre l'application mobile et les deux équipements du système. Ce format présente l'avantage d'être léger, lisible et facilement manipulable via la classe `QJsonObject` de Qt.

L'application communique avec deux équipements distincts, chacun ayant son propre jeu de trames.

Trames échangées avec l'ESP32 :

Action	Sens	Trame JSON
Obtenir les calendriers	App → ESP32	<code>{"action":"obtenirCalendrier"}</code>
Ajouter un calendrier	App → ESP32	<code>{"action":"ajouterCalendrier","jours":21,"hd":8,"md":0,"hf":17,"mf":0}</code>
Supprimer un calendrier	App → ESP32	<code>{"action":"supprimerCalendrier","id":1}</code>
Marche forcée	App → ESP32	<code>{"action":"marcheForcee","activer":true}</code>
Réponse calendrier	ESP32 → App	<code>{"action":"calendrier","id":1,"jours":21,"hd":8,"md":0,"hf":17,"mf":0}</code>
Alerte	ESP32 → App	<code>{"action":"alerte","type":"temperature"}</code>

Pour les calendriers, les jours de la semaine sont encodés sous forme de masque binaire afin de réduire la taille des trames. Chaque bit correspond à un jour : le bit 0 représente le lundi (valeur 1), le bit 1 le mardi (valeur 2), le bit 2 le mercredi (valeur 4), et ainsi de suite jusqu'au dimanche (valeur 64). Par exemple, un masque de valeur 21 correspond à Lundi + Mercredi + Vendredi (1 + 4 + 16).

Trames échangées avec le Raspberry Pi :

Action	Sens	Trame JSON
Obtenir les bornes	App → Raspi	{"action":"obtenirStation"}
Obtenir les véhicules	App → Raspi	{"action":"obtenirVehicule"}
Ajouter un véhicule	App → Raspi	{"action":"ajouterVehicule","name":"Renault Zoé","km":12000}
Modifier le kilométrage	App → Raspi	{"action":"modifierKilometrage","id":1,"km":13500}
Supprimer un véhicule	App → Raspi	{"action":"supprimerVehicule","id":1}
Réponse borne	Raspi → App	{"action":"station","id":1,"name":"Borne A","kwh":"22","status":"Disponible","ip":"192.168.1.50"}
Réponse véhicule	Raspi → App	{"action":"vehicule","id":1,"name":"Renault Zoé","km":12000}

5.2 WebSocket

Le protocole WebSocket permet d'établir une connexion persistante et bidirectionnelle entre deux équipements réseau. Contrairement aux requêtes HTTP classiques qui fonctionnent sur un modèle requête/réponse — où le client doit interroger régulièrement le serveur pour savoir si des données sont disponibles — le WebSocket maintient la connexion ouverte en permanence, ce qui permet à chaque équipement d'envoyer des données à tout moment, dans les deux sens, sans attendre d'être sollicité.

Ce fonctionnement correspond parfaitement aux besoins du projet TW2. En effet, l'application doit recevoir en permanence des informations en provenance des équipements sans action de l'utilisateur : l'ESP32 envoie régulièrement l'état de la borne, la puissance consommée et les éventuelles alertes, tandis que le Raspberry Pi notifie l'application dès qu'un changement d'état d'une borne est détecté. Un mécanisme de polling HTTP aurait introduit un délai entre chaque interrogation et une charge réseau inutile, rendant l'interface moins réactive. Le WebSocket élimine ces inconvénients en garantissant une transmission instantanée des données dès qu'elles sont disponibles.

Dans le projet TW2, l'application maintient deux connexions WebSocket simultanées et indépendantes : une vers le Raspberry Pi sur le port 8080, établie dès le démarrage de l'application, et une vers l'ESP32 sur le port 5555, établie dynamiquement lorsque le conducteur sélectionne une borne. L'adresse IP de l'ESP32 n'étant pas fixe, elle est fournie par le Raspberry Pi dans la trame de réponse de la borne, ce qui permet à l'application de se connecter à la bonne borne sans configuration manuelle.

En cas de déconnexion — par exemple lors de la mise en veille du téléphone ou d'une coupure réseau — un timer de reconnexion automatique se déclenche toutes les 5 secondes pour rétablir la connexion sans intervention de l'utilisateur.

6. Outils

6.1 Qt Creator

Qt Creator est l'environnement de développement intégré utilisé pour concevoir l'application mobile TW2. Qt est un framework multiplateforme développé par The Qt Company, permettant de créer des applications pour Android, iOS, Windows et Linux à partir d'une base de code commune. Dans ce projet, il est utilisé pour développer l'application Android en combinant du C++ pour la logique métier et du QML pour l'interface utilisateur.

Les classes Qt suivantes sont utilisées dans le projet :

QWebSocket permet d'établir et de gérer les connexions WebSocket vers le Raspberry Pi et l'ESP32. C'est la brique centrale de communication de l'application — toutes les trames JSON sont envoyées et reçues via cette classe.

QJsonObject / QJsonDocument permettent de construire et d'analyser les trames JSON échangées avec les équipements. QJsonDocument sert à sérialiser un objet JSON en texte pour l'envoi, et à désérialiser le texte reçu pour en extraire les données.

QTimer est utilisé pour déclencher les tentatives de reconnexion automatique toutes les 5 secondes en cas de perte de connexion WebSocket, que ce soit vers le Raspberry Pi ou vers l'ESP32.

QObject / Q_PROPERTY constituent la base du système de liaison entre le C++ et le QML. Les propriétés `activeBorne` et `activeVehicule` de la classe `CommunicationEsp` sont exposées au QML via `Q_PROPERTY`, ce qui permet à l'interface de se mettre à jour automatiquement lorsque ces valeurs changent.

QDebug est utilisé tout au long du développement pour afficher les trames envoyées et reçues dans la console via `adb logcat`, ce qui constitue l'outil de débogage principal puisque l'application tourne sur un appareil Android physique.

3.2 QML

QML (Qt Modeling Language) est le langage déclaratif utilisé pour concevoir l'interface utilisateur de l'application. Il permet de décrire l'apparence et le comportement des éléments graphiques de manière concise, et de les lier directement aux propriétés et signaux des classes C++.

Dans le projet, chaque page de l'application correspond à un fichier QML distinct : `BootPage.qml` pour la liste des bornes, `VehiculePage.qml` pour la gestion des véhicules, `DashboardPage.qml` pour le tableau de bord, `SessionsPage.qml` pour les calendriers de charge et `StatusPage.qml` pour le suivi en temps réel. Les fonctions utilitaires propres à chaque page sont externalisées dans des fichiers JavaScript associés (`BootPage.js`, `StatusPage.js`) afin de ne pas alourdir les fichiers QML.

6.3 GitHub

GitHub est utilisé comme plateforme de gestion de version tout au long du projet. Il permet de conserver un historique complet des modifications apportées au code, de collaborer avec les autres membres de l'équipe et de sauvegarder le projet en ligne.

Dans le cadre du Gestionnaire de charge, chaque membre de l'équipe travaille sur sa propre branche. La documentation Doxygen générée à partir des fichiers .h et .cpp est publiée automatiquement sur GitHub Pages via la commande `git subtree push`, ce qui permet aux professeurs et au jury d'accéder à la documentation technique en ligne.

7. Architecture de l'application

L'application Gestionnaire de charge est structurée autour de trois couches distinctes qui communiquent entre elles via le système de signaux et slots de Qt.

La **couche communication** est constituée des deux classes C++ `CommunicationEsp` et `CommunicationRaspi`. Chacune gère de manière autonome sa connexion client WebSocket, l'envoi des requêtes JSON et la réception des réponses. Elles émettent des signaux vers la couche QML lorsque des données sont reçues.

La **couche logique** est représentée par le `main`, qui centralise l'état global de l'application. Elle expose au QML les propriétés partagées entre les pages : borne active, véhicule sélectionné, statut des connexions, données de session, compteur d'alertes. Elle joue le rôle de point de liaison entre les deux classes de communication et l'interface utilisateur.

La **couche interface** est composée des fichiers QML organisés en pages de navigation. Le fichier `Main.qml` est le point d'entrée de l'application — il instancie les objets C++ et les expose au QML, gère la navigation entre les pages et affiche la barre de statut de connexion fixe en bas de l'écran ainsi que la fenêtre d'alerte globale.

Le flux de navigation entre les pages suit un ordre défini au démarrage :

BootPage (liste des bornes) → **VehiculePage** (sélection du véhicule) → **DashboardPage** (tableau de bord) → **SessionsPage** ou **StatusPage**

8. Diagramme de classe

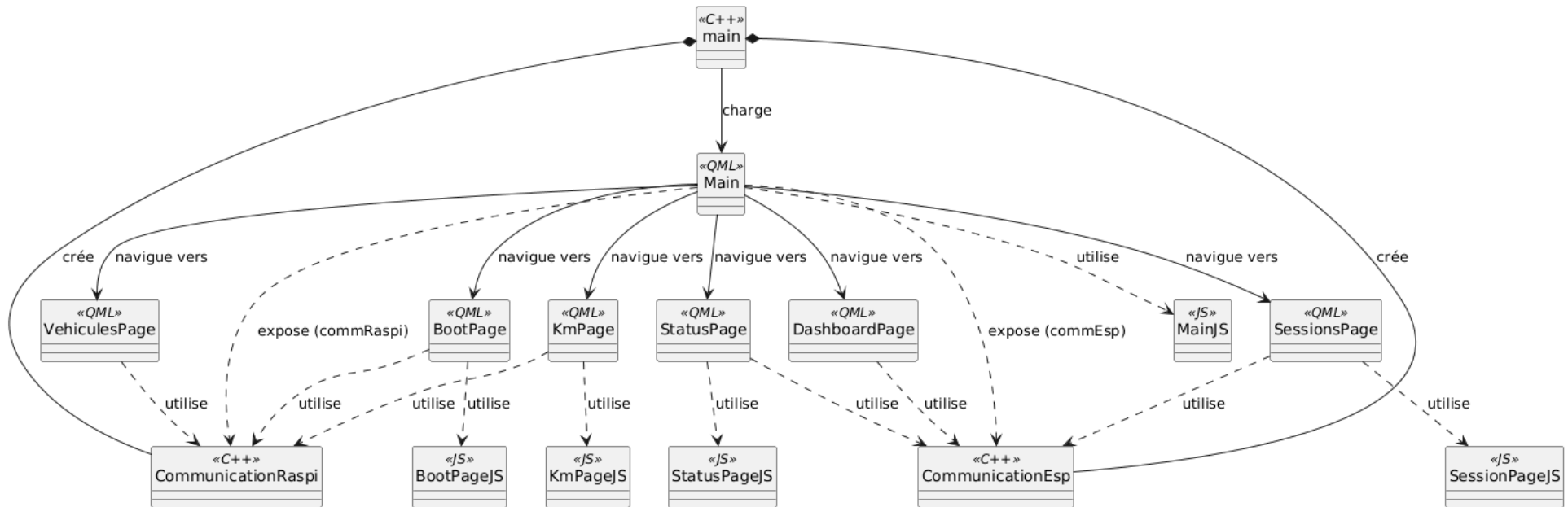


Figure 19: Diagramme de classe global

CommunicationEsp
-m_webSocket : QWebSocket -m_reconnectTimer : QTimer -m_url : QString -m_activeBorne : QString -m_activeVehicule : QString -m_attenteNouveauListe : bool
+connexion(ip : QString) : void +obtenirCalendrier() : void +ajouterCalendrier(jours : QString, start : QString, end : QString, resetListe : bool) : void +supprimerCalendrier(id : int) : void +marcheForcee(activer : bool) : void +activeBorne() : QString +setActiveBorne(borne : QString) : void +activeVehicule() : QString +setActiveVehicule(vehicule : QString) : void
~onConnected() : void ~onDisconnected() : void ~onTextMessageReceived(message : QString) : void ~tenterReconnexion() : void -joursVersmasque(jours : QString) : int -extraireHeureMinute(heureStr : QString, heure : int&, minute : int&) : void -masqueVersJours(masque : int) : QString -formaterHeure(heure : int, minute : int) : QString

CommunicationRaspi
-m_webSocket : QWebSocket -m_reconnectTimer : QTimer -m_url : QString
+obtenirStation() : void +obtenirVehicule() : void +ajouterVehicule(name : QString, km : QString) : void +modifierKilometrage(id : int, km : int) : void +supprimerVehicule(id : int) : void
~onConnected() : void ~onDisconnected() : void ~onTextMessageReceived(message : QString) : void ~tenterReconnexion() : void ~gererChangementEtat(etat : SocketState) : void

Figure 20: Diagramme de classe développé / précis

9. Fiche de test

Dans le cadre du projet TW2, une fiche de test unitaire a été réalisée afin de valider le fonctionnement de la classe `CommunicationEsp`, qui gère l'ensemble des échanges WebSocket entre l'application mobile et le gestionnaire de charge ESP32.

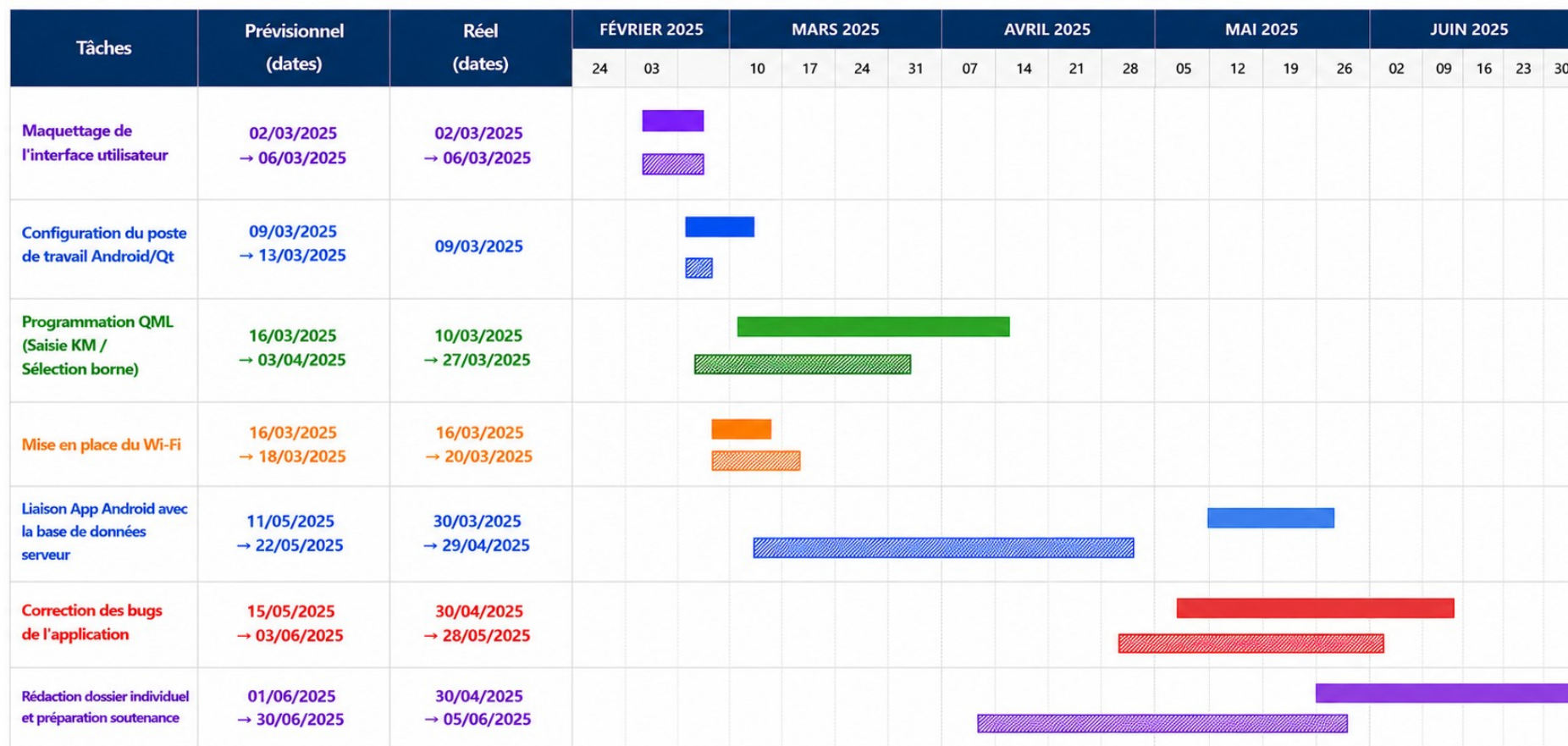
Le test a été effectué le 30 avril 2026 dans un environnement réel : un téléphone Android connecté au même réseau Wi-Fi que l'ESP32, relié au PC via USB pour permettre l'observation des trames JSON échangées en temps réel grâce à la commande `adb logcat | grep -E "\[ESP32\]"`.

Ce test couvre le cycle complet de gestion d'un calendrier de charge : programmation, modification et suppression, ainsi que le déclenchement de la marche forcée. Il permet de vérifier que les trames JSON envoyées par l'application sont bien reçues et interprétées par l'ESP32, et que les réponses de l'ESP32 sont correctement traitées et affichées dans l'interface.

<u>Référence du module testé</u> : communicationesp		FN°ETu.N°Test
Date du test :	30 Avril 2026 14h-17h	
Type du test :	Test unitaire	
Objectif du test :	Vérifier la programmation, la modification et la suppression d'un calendrier de charge via la communication WebSocket entre l'application mobile et le gestionnaire de charge ESP32, en observant les trames JSON échangées via adb logcat.	
<u>Conditions du test</u>		
État initial L'application mobile est fermée. Le téléphone est branché en usb sur le pc pour voir les logs PC avec un terminal ouvert pour les logs avec la commande adb logcat grep -E "[ESP32]".		Environnement du test Téléphone Android connecté sur le même réseau local (wifi). Application développée en C++ et QML
<u>Procédures du test</u>		
N°	Opérations	Résultats attendus
1	Lancer l'application mobile sur le téléphone.	L'application démarre et affiche la liste des bornes disponibles
2	Sur l'application mobile : Sélectionner la borne (Mr Cruchet).	L'application mobile passe sur la page de sélection des véhicules et la connexion au gestionnaire de charge apparaît sur le terminal
3	Sélectionner une voiture et entrer le kilométrage du véhicule et valider	Le kilométrage est validé et l'application passe sur la page d'accueil
4	Sélectionner l'onglet session	Le gestionnaire de charge envoie un calendrier vide qui s'affiche dans le terminal pour informer l'application qu'il est normal qu'aucun calendrier de charge n'apparaisse car aucun n'est enregistré pour l'instant
5	Sélectionner programmer une charge	L'application passe a la page suivante pour le choix de l'heure et des jours de programmation

6	Choisir les jours de charge que vous souhaitez ainsi que les heures puis appuyer sur valider	Le calendrier est transmis au gestionnaire de charge la trame apparaît sur le terminal et apparaît dans l'application
7	Sélectionner la charge que vous avez programmée en appuyant dessus	Vous revenez sur la page de sélection des jours et des heures
8	Modifier ce que vous souhaitez et cliquer sur enregistrer	Dans le terminal apparaissent les trames du nouveau calendrier de charge et dans l'application l'ancien calendrier est remplacé par le nouveau
9	Supprimer le calendrier programmée en appuyant sur la poubelle à côté de celui-ci	Un pop-up de confirmation apparaît
10	Valider la suppression dans le pop-up	Le calendrier est supprimé, la commande de suppression d'affiche dans le terminal et le calendrier est retiré sur l'application mobile
11	Revenez dans le menu et sélectionner l'onglet status	L'onglet status s'affiche
12	Appuyer sur le bouton lancer la charge	La trame de démarrage de la charge s'affiche dans le terminal (activer : true (ON))

10. Diagramme de Gant



■ Prévisionnel ■ Réel

Les dates sont incluses (du jour de début au jour de fin).

11. Compétences acquises

La réalisation du projet TW2 — gestionnaire de charge de véhicules électriques — m'a permis d'acquérir et d'approfondir un large panel de compétences techniques et professionnelles, en phase avec les réalités du milieu des systèmes numériques et embarqués.

En matière technique, le projet m'a permis :

- De concevoir et développer une application mobile Android complète en Qt/QML, en combinant la logique métier en C++ et l'interface utilisateur en QML.
- De mettre en œuvre le protocole WebSocket pour assurer une communication bidirectionnelle et temps réel entre l'application mobile, le Raspberry Pi et les bornes ESP32.
- D'utiliser et intégrer le format d'échange JSON via les classes QJsonObject et QJsonDocument pour structurer l'ensemble des trames échangées avec les équipements.
- De concevoir un système de reconnexion automatique robuste grâce à QTimer, garantissant la continuité de service en cas de perte de connexion.
- D'exploiter le système de signaux et slots de Qt ainsi que les propriétés Q_PROPERTY pour assurer la liaison dynamique entre le C++ et l'interface QML.
- De déployer et déboguer une application sur un appareil Android physique via adb, en exploitant les logs temps réel avec la commande adb logcat.
- De générer une documentation technique complète avec Doxygen.

D'autre part, au niveau professionnel, ce projet m'a également permis :

- D'appliquer une gestion de projet rigoureuse avec une répartition claire des tâches au sein d'une équipe de quatre membres, chacun responsable d'une brique du système.
- De travailler en intégration continue avec les autres membres de l'équipe, en adaptant l'application aux évolutions du firmware ESP32 et du serveur Raspberry Pi.
- De développer un esprit d'analyse et de résolution de problèmes concrets, notamment face aux problèmes de synchronisation des trames WebSocket et aux contraintes du réseau du lycée.
- De documenter rigoureusement les choix techniques, les protocoles d'échange et les procédures de test.

Pour résumer, ce projet constitue une expérience complète ayant permis de mobiliser les compétences du référentiel BTS CIEL IR et de se confronter à des problématiques techniques proches du milieu professionnel.

12. Perspectives d'évolution

Le projet TW2 est fonctionnel dans son état actuel, mais plusieurs axes d'amélioration ont été identifiés au cours du développement et pourraient faire l'objet d'évolutions futures.

Notifications en arrière-plan

Actuellement, les alertes émises par l'ESP32 (surchauffe, surcharge) ne sont visibles que lorsque l'application est ouverte au premier plan. Une évolution naturelle serait d'implémenter un système de notifications Android natives, permettant d'avertir le conducteur même lorsque l'application est en arrière-plan ou verrouillée. Cette fonctionnalité a été conçue et partiellement développée via une interface JNI entre le C++ Qt et une classe Java NotificationHelper, mais n'a pas pu être intégrée dans le délai imparti en raison de la complexité de l'interfaçage et des priorités de stabilisation de l'application. Dans une version future, cette évolution permettrait une surveillance passive de la borne sans que le conducteur ait besoin de garder l'application ouverte en permanence, ce qui représente un gain significatif en termes d'ergonomie et de sécurité.

Sécurisation des communications

Les échanges WebSocket entre l'application et les équipements sont actuellement non chiffrés, ce qui constitue une limite importante dans un contexte de déploiement réel. N'importe quel appareil connecté au même réseau pourrait en théorie intercepter les trames JSON échangées ou envoyer des commandes frauduleuses à une borne. Une première évolution consisterait à passer en WebSocket sécurisé (WSS) avec un certificat TLS, afin de chiffrer l'ensemble des communications. Qt propose la classe QSslConfiguration pour mettre en œuvre ce chiffrement de manière transparente, sans modifier la logique applicative existante. En complément, un mécanisme d'authentification par jeton pourrait être ajouté : à la connexion, l'application transmettrait un identifiant unique que l'ESP32 ou le Raspberry Pi vérifierait avant d'accepter toute commande. Ces deux mesures combinées garantiraient l'intégrité et la confidentialité des échanges.

Historique des sessions de charge

Actuellement, les données de session (énergie consommée, durée, coût estimé) sont affichées en temps réel sur la page Status mais ne sont pas conservées. Une évolution utile serait d'enregistrer chaque session terminée dans la base de données MySQL du Raspberry Pi, puis de les rendre consultables depuis l'application mobile sous forme d'historique. Le conducteur pourrait ainsi suivre sa consommation sur le long terme, identifier ses habitudes de recharge et estimer ses coûts mensuels. Cette fonctionnalité nécessiterait l'ajout d'une nouvelle trame JSON entre l'application et le Raspberry Pi, ainsi qu'une nouvelle page QML dédiée à la consultation de l'historique.

Gestion multi-utilisateurs

Dans sa version actuelle, l'application ne gère pas de notion d'utilisateur : toute personne connectée au réseau peut piloter n'importe quelle borne. Dans un contexte de déploiement en entreprise ou en copropriété, il serait pertinent d'introduire un système de comptes utilisateurs

avec authentification. Chaque conducteur disposerait d'un profil associant ses véhicules et ses préférences de charge. Le Raspberry Pi jouerait alors le rôle de serveur d'authentification, et l'application afficherait uniquement les bornes et les calendriers associés au compte connecté. Cette évolution impliquerait une refonte partielle de la base de données et des protocoles d'échange, mais rendrait le système beaucoup plus adapté à un usage multi-utilisateurs réel.